

Day1 · 4차시

데이터 영속화 & 배포 & 디자인 업그레이드

내 손의 프로토타입을, 인터넷 주소로 — 예쁘게 (Codex CLI 단독)

오늘의 문제 3가지 → 해법 3가지

▶ 지금 앱의 한계를 하나씩 지운다 — 전원 전부 실습

① 새로고침하면 사라진다

저장: localStorage → **Supabase** (클라우드 DB)

② 내 컴퓨터에서만 열린다

배포: **GitHub Pages + Vercel** — 공개 URL 2개

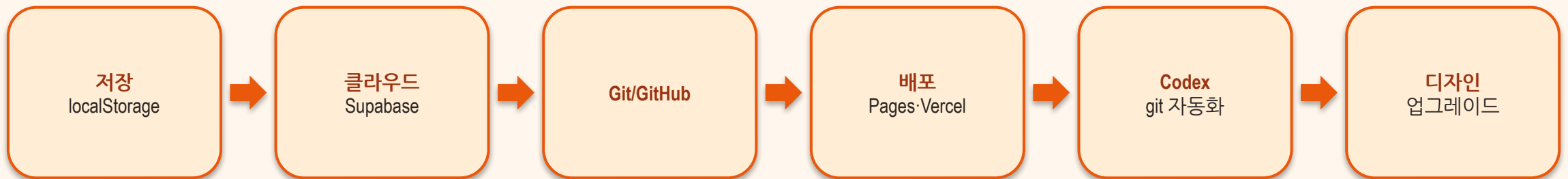
③ 솔직히 좀 밋밋하다

디자인: 프롬프트 · frontend 스킬 · 3D 배경

끝 그림 — 오늘의 결과물 체크리스트

▶ 저장 기능 추가·커밋·배포·디자인까지 Codex CLI 하나로

- Supabase에 저장되는 데이터 — 다른 브라우저에서도 보인다
- 공개 URL 2개 — `github.io`(Pages) + `vercel.app`(Vercel)
- 디자인 업그레이드 1회 이상 — 전부 프롬프트 한두 줄



SECTION 01

데이터를 남긴다 — localStorage → Supabase

localStorage — 브라우저 안의 작은 메모장

▶ 왜 사라지나: 데이터가 메모리(변수)에만 → 새로고침 = 초기화

- 브라우저 안에 저장 — 창을 닫았다 열어도 남는다 (가입·카드 0)
- 한계: 그 컴퓨터·그 브라우저에만 — 공유 불가

Codex

"새로고침해도 데이터가 남도록 localStorage에 저장하고, 로드 시 불러오게 해줘"

localStorage의 벽 — 클라우드 DB가 필요한 순간

▶ 친구 폰에서 열면 빈 화면 — 데이터가 내 브라우저 안에만 있으니까

localStorage

- 내 브라우저에만 저장
- 기기 간 공유 불가
- 가입 0 · 데모용
- 새로고침은 버틴다

VS

Supabase (클라우드 DB)

- 인터넷 위의 진짜 DB (Postgres)
- 어디서 열어도 같은 데이터
- 서버 코드 없이 브라우저에서 읽고 쓰기
- 무료로 시작 · 카드 등록 불필요

Supabase 시작 3단계 — 웹 콘솔에서

▶ 관리형 Postgres + 자동 생성 REST API



표 = 엑셀 시트와 같은 구조 (id·text·created_at). 복사한 URL·key 두 줄만 있으면 준비 끝.

CDN 한 줄 — 정적 HTML에서 바로 읽고 쓴다

▶ 설치·빌드 없음 (supabase-js 공식 CDN)

```
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>
<script>
  const db = supabase.createClient("https://내프로젝트.supabase.co", "sb_publishable_...");
  await db.auth.signInAnonymously(); // 익명 로그인 (INSERT 정책 통과)
  await db.from("memos").insert({ text: "점심: 김치찌개" }); // 저장
  const { data } = await db.from("memos").select(); // 불러오기
</script>
```

Codex

"localStorage 대신 Supabase에 저장하게 바꿔줘. URL은 ○○, publishable key는 ○○, 테이블은 memos(text 컬럼)야. 로드 시 signInAnonymously로 익명 로그인 먼저"

공개 전 설정 3가지 — 정책 2개 · 익명 로그인 · GRANT

▶ 정책은 템플릿 그대로 · 새 프로젝트는 테이블 권한 기본 잠금이라 GRANT 1회 필요

정책 ① SELECT

템플릿 "Enable read access for all users" — public, 그대로 Save

정책 ② INSERT

템플릿 "Enable insert for authenticated users only" — authenticated, 그대로 Save

Auth 설정

Allow anonymous sign-ins ON + 코드에 signInAnonymously() 한 줄

GRANT — SQL 1회

403 "permission denied for table" 방지 — 아래 3줄 Run

```
grant usage on schema public to anon, authenticated;
grant select on table public.memos to anon, authenticated;
grant insert on table public.memos to authenticated;
```

Supabase 무료 한도 (2026)

▶ 캠프 데모엔 충분 — 출처: supabase.com/pricing

DB 500MB

파일 1GB · 월 활성 사용자 5만 · 프로젝트 2개

무료 시작

가입만 하면 카드 등록 불필요

⚠ 1주 미사용 시 일시정지

콘솔에서 클릭 한 번으로 재개 — 시연 전날 열어둘 것

SECTION 02

배포 전에 알아둘 단어 — Git & GitHub

Git · GitHub · 레포 · 커밋 · 푸시 — 한 줄씩

▶ 명령어는 외우지 말고 개념만 — 실행은 Codex가 대신한다

Git

파일의 타임머신/세이브 — 변경 이력을 단계별로 저장, 되돌리기 가능

GitHub

코드용 구글드라이브 + 웹 호스팅 — 인터넷에 올려 백업·공유·공개

레포(Repository)

프로젝트 폴더 한 칸 — 파일 + 변경 이력이 함께

커밋 / 푸시

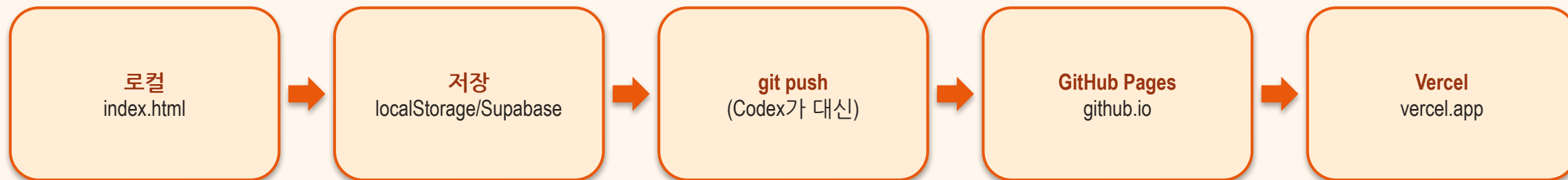
저장 지점 기록 / 내 PC → GitHub 업로드

GitHub Pages

올린 index.html을 무료 웹주소로 공개 — 오늘의 배포 ①

한 줄 그림 — 로컬에서 공개 URL까지

▶ 내 컴퓨터에만 있던 앱을, 전 세계가 보는 주소로



SECTION 03

배포 ① — GitHub Pages

레포 만들기 → 파일 올리기

▶ github.com/new → 이름 → Public → Create · 대문 파일명은 반드시 index.html

```
git init  
git add .  
git commit -m "first deploy"  
git branch -M main  
git remote add origin https://github.com/<아이디>/<레포>.git  
git push -u origin main
```

첫 push 때 GitHub 로그인(브라우저/토큰). 외우기 싫으면 → Codex에 위임 (SECTION 05).

명령어, 한 줄씩 무슨 뜻

▶ 여섯 줄이 하는 일 전부

git init

이 폴더를 Git이 관리하도록 시작 — 폴더당 맨 처음 한 번만

git add .

바뀐 파일들을 저장할 목록에 담기 (.= 폴더 전체)

git commit -m

담은 파일을 하나의 저장 지점으로 기록 — 따옴표는 저장 메모

git branch -M main

기본 줄기 이름을 main으로 (GitHub 표준)

git remote add origin

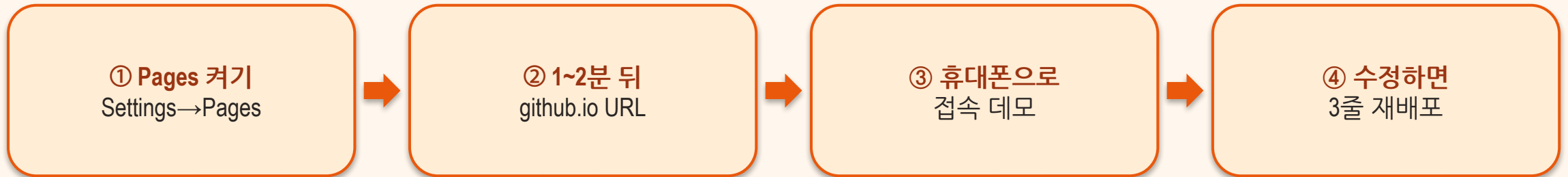
내 PC ↔ GitHub 레포 연결 (origin = 주소 별명)

git push -u origin main

기록을 GitHub로 업로드 — 이후엔 git push 만으로

Pages �키기 → 공개 주소 확인 → 재배포

▶ Settings → Pages → Deploy from a branch → main / (root) → Save



```
git add . && git commit -m "update" && git push # 1~2분 후 자동 반영
```

SOS 배포 문제 해결

▶ 실습 중 가장 자주 보게 될 표

사이트가 404

파일명 **index.html** 인지 · Pages Branch=main / root 확인

빈 화면 / 옛 화면

1~2분 더 기다린 뒤 강력 새로고침 (Cmd/Ctrl+Shift+R)

push 인증 실패

비밀번호 대신 토큰 또는 `gh auth login`

Branch 선택지 없음

먼저 main 브랜치로 1회 push 되어야 목록에 나타남

이미지/CSS 안 보임

상대경로(./style.css) · 대소문자 정확히

SECTION 04

배포 ② — Vercel

Vercel — 폴더를 던지면 URL이 나온다

▶ 결과는 Pages와 같은 공개 URL — 반영 수 초 · push마다 자동 재배포

① 웹 드래그&드롭

vercel.com → Add New → Project → 폴더 끌어다 놓기

② CLI

npm i -g vercel → 앱 폴더에서 vercel (첫 회 이메일 로그인)

③ GitHub 연동 — 오늘의 방법

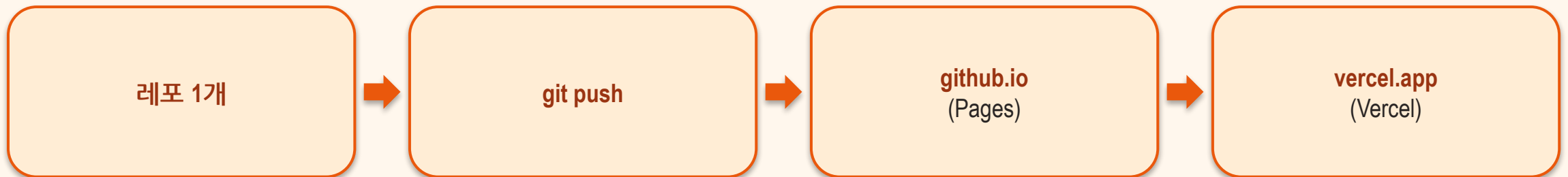
Pages에 쓴 레포 그대로 import → 이후 git push만 하면 자동 재배포

실습 — 같은 레포로 두 번째 URL

▶ git push 한 번 → Pages·Vercel 둘 다 갱신

Codex

"이 폴더를 vercel CLI로 배포해줘. 로그인에 필요하다면 절차를 알려줘"



Vercel 무료(Hobby) 한도 (2026)

▶ 정적 앱엔 넉넉 — 출처: vercel.com/docs/limits

대역폭 100GB/월

프로젝트 200개 · 배포 100회/일 · CLI 업로드 100MB

초과 시 30일 잠금

과금 대신 기능 잠금 — 요금 폭탄 없음

⚠ 상업용 불가

Hobby 약관 — 캠프·포트폴리오는 OK

SECTION 05

명령 대신 말로 — Codex git 자동화

Codex로 git에 올려보기

▶ 명령을 몰라도 — Codex가 `init→add→commit→push`를 스스로 만들어 실행

Codex ①

"이 폴더를 git 저장소로 초기화하고, 모든 파일을 "first deploy" 메시지로 커밋해줘"

Codex ②

"원격 저장소 `https://github.com/<아이디>/<레포>.git` 에 연결하고 main으로 push 해줘"

실행 전 승인을 묻으면 내용 확인 후 허용. 첫 push 때 GitHub 로그인 안내를 따르면 끝.

수정할 때마다 자동 commit·push

▶ Codex 자체엔 자동 커밋 미지원 — 우회 2가지

A. AGENTS.md 규칙

- "수정하면 항상 add→commit→push" 규칙 추가
- codex --full-auto 로 승인 생략
- 터미널 흐름 유지
- 작업(프롬프트) 단위 — 100% 보장은 아님

VS

B. GitDoc 확장 가장 쉬움

- VS Code 확장 GitDoc 설치
- GitDoc: Enable
- 저장(Cmd/Ctrl+S)마다 자동 commit+push
- 커밋이 자주 쌓임 — 데모·개인용엔 무방

SECTION 06

예쁘게 만든다 — 디자인 업그레이드

왜 맛있는가 — "AI 기본값"의 함정

▶ 그냥 시키면 어디서 본 듯한 화면(AI-slop)이 나온다

그냥 시키면

- 흔한 시스템 폰트
- 보라 그라데이션
- 카드 나열 레이아웃
- "어디서 봤더라" 느낌

VS

디자인 결정을 주면

- 색·폰트·레이아웃을 내가 결정
- 포인트색 1개 · 서체 2개
- 여백과 타이포로 위계
- 명확한 제약일수록 잘 그린다

방법 ① — 프롬프트에 디자인 결정을 담아라

▶ 모델은 구체적인 제약을 줄수록 잘 그린다 (OpenAI frontend 가이드)

Codex

"다크 테마로, 포인트 색은 #FF6B35 하나만. 제목은 큰 세리프, 본문은 산세리프. 카드 나열 대신 여백과 큰 타이포그래피로 위계를 잡아줘. 기능은 그대로."

핵심: 형용사("예쁘게")가 아니라 **결정**(색 코드·서체·레이아웃 규칙)을 준다.

방법 ② — frontend 디자인 스킬 설치

▶ OpenAI 공식 skills 카탈로그 (github.com/openai/skills) — 취향을 시스템으로

- 2차시에서 배운 스킬 설치 절차 그대로 — 설치하면 매번 길게 안 적어도 기본값이 달라진다
- 흔한 폰트 금지 · 서체 2개 · 포인트색 1개 제한 · 카드 남발 금지
- 코드를 쓰기 전에 비주얼 컨셉부터 쓰게 강제

Codex

"frontend 스킬을 적용해서 이 앱의 디자인을 다시 잡아줘. 기능은 그대로."

방법 ③ — 3D 배경 한 스펀: Vanta.js

▶ three.js 기반 · 스크립트 2줄 · 효과 미리보기: vanta.js.com

```
<div id="vanta-bg" style="position:fixed;inset:0;z-index:-1"></div>    <!-- body 맨 앞 -->
<script src="https://cdnjs.cloudflare.com/ajax/libs/three.js/r134/three.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/vanta/dist/vanta.waves.min.js"></script>
<script>VANTA.WAVES({ el: "#vanta-bg" })</script>    <!-- 셋 다 </body> 직전, 이 순서 -->
```

프롬프트

"Vanta.js WAVES 3D 배경을 추가해줘. body 맨 앞에 <div id="vanta-bg" style="position:fixed;inset:0;z-index:-1"></div>를 만들고, three.js(r134)→vanta.waves CDN→VANTA.WAVES({el:"#vanta-bg"}) 스크립트 3개를 이 순서로 </body> 직전에 넣어줘. body 컨테이너 배경은 투명하게 해서 배경이 보이게 하고, 글자 가독성 유지"

안 보이는 3대 원인: head에서 init / #vanta-bg 없음 / 불투명 배경이 덮음 — 구조를 정확히 지시.

절제 3원칙 — 그리고 재배포

▶ 3D를 넣었다고 다 예뻐지지 않는다 — 절제가 곧 고급스러움

① wow 요소는 한 곳만

배경이면 배경만 — 두 개부터는 촌스러워진다

② 서체 2개 · 포인트색 1개

제목용 + 본문용, 색은 하나로 통일

③ 움직임은 은은하게

멀미 유발 금지 — 콘텐츠보다 튀면 실패

```
git add . && git commit -m "design" && git push # Pages·Vercel 둘 다 자동 갱신
```

SECTION 07

무료 플랜 & 마무리

오늘 쓴 서비스, 전부 무료로 시작

▶ 카드 등록 없이 — 수치는 강의 직전 재확인

GitHub Pages

정적 호스팅 · HTTPS 자동 — 정적 파일만 · Public 레포 권장

Supabase

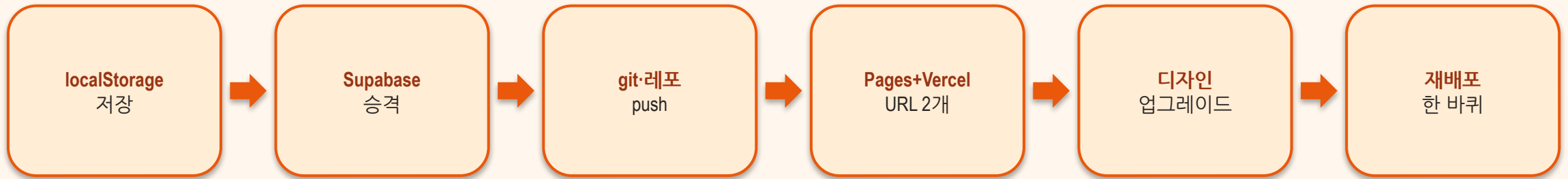
DB 500MB · MAU 5만 — △ 1주 미사용 시 일시정지

Vercel (Hobby)

대역폭 100GB/월 · 배포 100회/일 — 상업용 불가 · 초과 시 30일 잠금

오늘 한 바퀴 — 체크리스트

▶ 기획(PRD) → 스펙 → 구현 → 저장 → 배포 → 디자인



- □ Supabase 저장 — 다른 브라우저에서도 데이터 확인 □ 공개 URL 2개 휴대폰 접속
- □ 디자인 1가지 이상 업그레이드 □ 수정 → push → 두 배포 자동 갱신 확인

클로징 — 다음은 팀이다

▶ 1차시 점심 추천기 → 4차시 공개 URL까지, 하루에 한 길

- 오늘 만든 주소를 공유하세요 — 데이터가 남고, 디자인이 다른 내 앱
- 오늘 저녁 팀 프로젝트 Day 1: 기획 → 개발 → 첫 배포 — 팀 서비스로 한 사이클 더